Математическое моделирование

Лабораторная работа №5
«Сети Петри»

Спецификация программы: теория, разбор двух сетей, код

Содержание

1 Обзор 2
1.1 Что требует задание 2
1.2 Что делает программа 2
1.3 Стек и запуск 2
1.4 Модули 2
1.5 Поток данных 3
2 Теория 4
2.1 Сеть Петри 4
2.2 Маркировка 4
2.3 Правило срабатывания 4
2.4 Конфликт и параллельность 4
2.5 Диаграмма маркировок 4
2.6 Свойства сети 5
2.7 Классы сетей 5
3 Две сети из задания 6
3.1 Правая сеть 6
3.2 Левая сеть 7
4 Как программа решает задачу 9
4.1 Сеть и маркировка: `net.rs` 9
4.2 Диаграмма маркировок: `graph.rs` 9
4.3 Текстовый ввод: `parse.rs` 10
4.4 Класс и свойства: `analysis.rs` 10
4.5 Окно: `canvas.rs` и `app.rs` 11
5 Тесты 12
6 Допущения и ограничения 13

1 Обзор

1.1 Что требует задание

В `ММод ЛР №5.pdf` (в самом файле работа названа «№4») нужно смоделировать две сети Петри с рисунка 1 и исследовать их:

1. найти характеристики сети и сравнить с теоретическими;
2. проверить, достижима ли заданная маркировка из начальной, то есть входит ли она в диаграмму маркировок;
3. проверить, возможно ли параллельное срабатывание нескольких переходов;
4. определить, к какому классу относится сеть.

Готовые средства (pntool для Matlab, сайт `petri.hp102.ru`) разрешены, а собственное приложение даёт дополнительные баллы. Здесь написано собственное приложение.

> Схему сетей я прочитал по картинке из PDF (рис. 1 в задании). Если на вашем экземпляре стрелки идут иначе, нужно поправить два текста в `lab5/src/nets.rs` (`LEFT_TEXT` и `RIGHT_TEXT`), остальное пересчитается само.

1.2 Что делает программа

Программа `lab5` — окно с одной сетью Петри.

- Слева текстовое поле: сеть задаётся текстом. Кнопки загружают обе сети из задания. Свою сеть можно ввести тем же форматом.
- В центре граф сети: позиции кружками с числом фишек, переходы чёрточками, разрешённые переходы зелёные. Клик по зелёному переходу срабатывает его, кнопки «Back» и «Reset» откатывают шаги. Узлы можно перетаскивать. Переключатель над холстом («Net» / «Marking graph») заменяет сеть диаграммой маркировок.
- Под полем текста результаты: класс сети, число вершин и рёбер диаграммы маркировок, границы фишек в позициях, безопасность, обратимость, живость, тупики, пары переходов, которые могут сработать параллельно, и проверка «достижима ли заданная маркировка».

1.3 Стек и запуск

Rust (edition 2024), `eframe`/`egui` для окна. Крейт `lab5` входит в cargo-workspace корня репозитория и использует `common` только ради цветов. Он состоит из библиотеки (всё, что считает) и бинарника (окно), поэтому тесты не зависят от графики.

```
cargo run -p lab5          # окно
cargo test -p lab5         # 31 тест
cargo clippy -p lab5 --all-targets
```

1.4 Модули

Файл	Роль
`net.rs`	Структуры `Net`, `Transition`, `Marking`; правило срабатывания и проверка параллельности.
`nets.rs`	Обе сети из задания: в виде кода (`left()`, `right()`) и в виде текста (`LEFT_TEXT`, `RIGHT_TEXT`).
`graph.rs`	Диаграмма маркировок (`ReachabilityGraph`), поиск пар параллельных переходов.
`parse.rs`	Разбор текстового описания сети и маркировки, сообщения об ошибках с номерами строк.
`analysis.rs`	Класс сети (`classify`), свойства по диаграмме (`analyze`).
`canvas.rs`	Круговая раскладка и рисование сети, клик и перетаскивание; послойная раскладка и рисование диаграммы маркировок.
`app.rs`	Окно: поле текста, симуляция, анализ.
`main.rs`	Запуск окна.

1.5 Поток данных

1. Текст сети попадает в `parse` и превращается в `Net` и начальную маркировку. Ошибки показываются под полем, последняя корректная сеть остаётся на экране.
2. По сети и маркировке `ReachabilityGraph::build` строит диаграмму маркировок (обход в ширину, лимит 10 000 вершин).
3. По диаграмме `analyze` считает границы, тупики, обратимость и живость. `classify` смотрит только на структуру сети, `concurrent_pairs` ищет параллельные пары.
4. Окно рисует сеть, а клик по разрешённому переходу вызывает `Net::fire`.

2 Теория

2.1 Сеть Петри

Сеть Петри — двудольный ориентированный граф. В нём два вида вершин:

- **позиции** (кружки) хранят **фишки**; они описывают условия или ресурсы;
- **переходы** (чёрточки) описывают события.

Дуги соединяют только позицию с переходом или переход с позицией. Формально сеть — это набор (P, T, F, M_0): множество позиций P, множество переходов T, дуги F и начальная маркировка M_0. Для перехода t множество $\text{вх}(t)$ — это его входные позиции (откуда идёт дуга в t), а $\text{вых}(t)$ — выходные.

В этой работе у каждой дуги кратность 1: переход забирает и кладёт по одной фишке. Дуги большей кратности программа не поддерживает и отвергает при вводе (раздел 4.3).

2.2 Маркировка

Маркировка M — это число фишек в каждой позиции, $M(p) \in \mathbb{N}$. Она описывает состояние сети целиком. Для правой сети начальная маркировка — вектор $(1, 0, 0, 0, 0, 1)$ по позициям $p_1, \ldots, p_6$: по фишке в p_1 и p_6.

2.3 Правило срабатывания

Переход t **разрешён** в маркировке M, если в каждой его входной позиции есть фишка:

$$M(p) \geq 1 \quad \text{для всех } p \in \text{вх}(t).$$

Срабатывание разрешённого перехода даёт новую маркировку M':

$$M'(p) = M(p) - [p \in \text{вх}(t)] + [p \in \text{вых}(t)],$$

то есть из каждой входной позиции забирается по фишке, в каждую выходную кладётся по фишке. Выражение в квадратных скобках равно 1, если условие верно, и 0 иначе. Если позиция входит и в $\text{вх}(t)$, и в $\text{вых}(t)$, её число фишек не меняется.

Срабатывание — одно целое действие. Переход с одним входом и тремя выходами (t_1 в правой сети) **не выбирает** один из выходов: фишка появляется во всех трёх. Это разветвление (fork). Переход с несколькими входами (t_4) ждёт, пока в каждом из них будет фишка. Это слияние (join).

2.4 Конфликт и параллельность

Если из одной позиции выходят несколько разрешённых переходов, они могут бороться за её фишки. В левой сети из p_4 выходят a и d, а фишка в p_4 одна. Сработает только один из них, и выбор **недетерминированный**. Это **конфликт**.

Переходы t и u могут сработать **параллельно** в маркировке M, если фишек хватает на обоих сразу, то есть для каждой позиции

$$M(p) \geq [p \in \text{вх}(t)] + [p \in \text{вх}(u)].$$

Это не то же самое, что «у переходов нет общих позиций». Общие **входные** позиции не мешают, если в них достаточно фишек (например, две фишки в p_4 левой сети хватило бы и a, и d). Общие **выходные** позиции не мешают никогда: оба перехода просто добавляют по фишке.

Достаточно проверять пары: если параллельно может сработать группа из трёх и более переходов, то любые два из неё тоже срабатывают параллельно.

2.5 Диаграмма маркировок

Диаграмма маркировок (граф достижимости) — граф, в котором

- вершины — маркировки, достижимые из M_0;
- рёбра $M \xrightarrow{t} M'$ — срабатывания разрешённых переходов t.

Вопрос «достижима ли маркировка M из начальной» значит «есть ли M среди вершин диаграммы». Строится диаграмма так же, как обход графа: начать с M_0, для каждой найденной маркировки применить все разрешённые переходы, новые маркировки добавить, уже найденные не повторять. Построение заканчивается, когда у всех найденных вершин все переходы разобраны и новых маркировок нет.

Если число фишек в какой-то позиции может расти бесконечно, диаграмма бесконечна, и обход не закончится. Поэтому в программе есть лимит на число вершин (по умолчанию 10 000). Обход, остановленный лимитом, **не доказывает** неограниченность: он говорит только, что граф не достроен за лимит.

2.6 Свойства сети

Все свойства читаются с полностью построенной диаграммы.

Свойство	**Определение**	**Как проверяется по диаграмме**
Ограниченность	Число фишек в каждой позиции не превосходит некоторого K ни в какой достижимой маркировке. При $K = 1$ сеть называется безопасной.	Диаграмма конечна. Граница K для позиции — максимум её значений по всем вершинам.
Тупик	Достижимая маркировка, в которой не разрешён ни один переход.	Вершина без исходящих рёбер.
Обратимость	Из любой достижимой маркировки можно вернуться в M_0.	Из каждой вершины есть путь в вершину M_0. Проверяется обратным обходом от M_0 по развёрнутым рёбрам: должны быть достигнуты все вершины.
Живость	Из любой достижимой маркировки любой переход можно когда-нибудь сделать разрешённым, возможно после цепочки других срабатываний.	Для каждого перехода t обратный обход стартует из вершин, где есть ребро t. Должны быть достигнуты все вершины.

Живость сильнее отсутствия тупиков. Тупиков может не быть, а какой-то переход всё равно «умирает» навсегда. Пример: токен входит в цикл между двумя позициями и больше не возвращается в начало; цикл крутится, но переход, который вёл в цикл, уже никогда не сработает. Этот случай проверяется в тесте (раздел 5).

2.7 Классы сетей

Классы задаются структурой сети: сколько входных и выходных дуг у позиций и переходов.

Класс	**Определение**
Конечный автомат (state machine)	У каждого перехода ровно один вход и один выход. Фишка идёт по сети, не размножаясь и не сливаясь. Конфликты есть, параллелизма нет.
Граф событий (marked graph)	У каждой позиции ровно один входной и один выходной переход. Конфликтов нет, а fork и join есть.
Сеть свободного выбора (free-choice)	Если у позиции несколько выходных переходов, то эта позиция — единственный вход каждого из них. Выбор не зависит от других позиций.
Общая сеть	Всё остальное: ограничений на структуру нет.

Классы вложены: конечные автоматы и графы событий входят в класс свободного выбора, а он входит в общие сети. Названия и границы немного различаются в учебниках, поэтому в методичке преподавателя классов может оказаться больше (например, расширенный свободный выбор и асимметричный выбор). Программа проверяет три перечисленных признака и называет самый узкий класс.

3 Две сети из задания

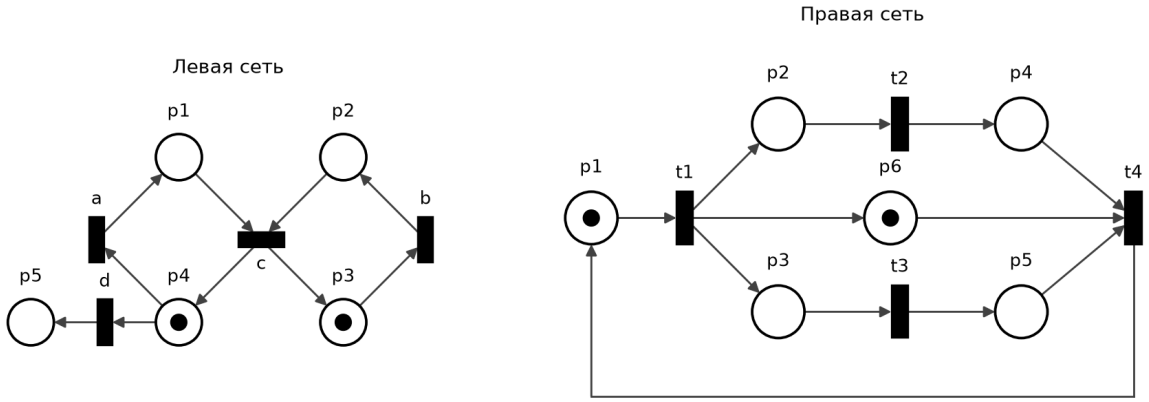


Рис. 1. Сети с рисунка 1 задания. Фишки в начальной маркировке показаны точками; число вместо точки означает несколько фишек.

В программе позиции нумеруются по первому появлению в тексте. Ниже везде маркировки записаны по порядку $p_1, p_2, \ldots$ из задания, а вершины диаграммы — номерами обхода в ширину (так же, как в программе: переходы перебираются в порядке их описания).

3.1 Правая сеть

Текст сети (он же загружается кнопкой «Load right net»):

```
t1: p1 -> p2 p3 p6
t2: p2 -> p4
t3: p3 -> p5
t4: p4 p5 p6 -> p1
marking: p1=1 p6=1
```

Это цикл «заказа»: t_1 запускает две параллельные работы ($p_2 \to t_2 \to p_4$ и $p_3 \to t_3 \to p_5$) и берёт «разрешение» p_6, t_4 завершает заказ, когда готовы обе работы, и возвращает его в p_1.

3.1.1 Диаграмма маркировок

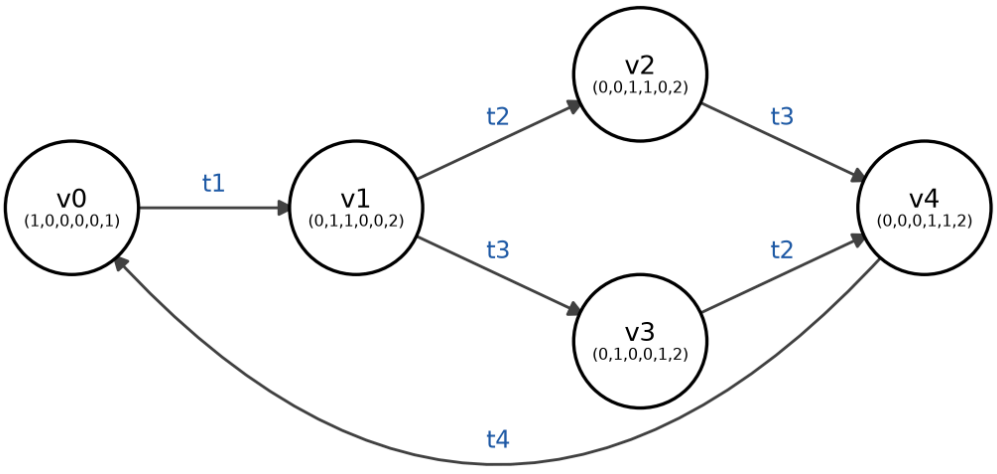


Рис. 2. Диаграмма маркировок правой сети: 5 вершин, 6 рёбер.

Вершина	Маркировка $(p_1, \ldots, p_6)$	Исходящие рёбра
v_0	$(1,0,0,0,0,1)$	$t_1 \to v_1$
v_1	$(0,1,1,0,0,2)$	$t_2 \to v_2$, $t_3 \to v_3$
v_2	$(0,0,1,1,0,2)$	$t_3 \to v_4$

v_3	$(0, 1, 0, 0, 1, 2)$	$t_2 \to v_4$
v_4	$(0, 0, 0, 1, 1, 2)$	$t_4 \to v_0$

Вершина v_4 достигается двумя путями (v_2 и v_3), это ромб из двух независимых работ. В диаграмме нет ни тупиков, ни лишних состояний.

3.1.2 Ответы на пункты задания

- **Достижимость.** Достижимы ровно маркировки из таблицы. Например, $(0, 0, 0, 1, 1, 2)$ достижима (вершина v_4), а $(1, 1, 0, 0, 0, 1)$ — нет.
- **Параллельность.** Единственный случай: в v_1 переходы t_2 и t_3 берут фишки из разных позиций (p_2 и p_3) и могут сработать одновременно. В остальных вершинах разрешён не более одного перехода, поэтому других пар нет.
- **Класс.** Это граф событий: у каждой из $p_1, \ldots, p_6$ ровно один входной и один выходной переход (p_1: $t_4 \to p_1 \to t_1$; p_6: $t_1 \to p_6 \to t_4$ и так далее). Конечным автоматом сеть не является: у t_1 три выхода, у t_4 три входа. Так как граф событий — частный случай свободного выбора, формально она и free-choice, но называть нужно самый узкий класс.
- **Характеристики.** Границы фишек: $p_6 - 2$, остальные позиции — 1, так что сеть ограничена, но не безопасна. Тупиков нет. Диаграмма сильно связная (из каждой вершины можно дойти до любой), значит сеть **обратима**. Каждый из $t_1, \ldots, t_4$ встречается на ребре, и диаграмма сильно связная, поэтому из любой вершины можно дойти до ребра любого перехода: сеть **жива**.

3.1.3 Сравнение с теорией

В задании не сказано, что считать теоретическими характеристиками. Для графа событий есть стандартные результаты, и с ними можно сравнить то, что нашла программа:

- В сильно связном графе событий начальная маркировка **жива** тогда и только тогда, когда в каждом ориентированном контуре есть хотя бы одна фишка. Все контуры правой сети проходят через t_1 и t_4, то есть через p_1, где фишка есть. Значит, сеть жива, как и показала диаграмма.
- Граница фишек в позиции равна минимуму суммы фишек по контурам, проходящим через эту позицию. Контур $p_1 \to t_1 \to p_6 \to t_4 \to p_1$ содержит 2 фишки (p_1 и p_6), поэтому в p_6 не больше 2. Контуры через p_2, p_3, p_4, p_5 содержат по одной фишке (в p_1), поэтому там не больше 1. Это совпадает с границами из диаграммы.

3.2 Левая сеть

Текст сети (кнопка «Load left net»):

```
a: p4 -> p1
b: p3 -> p2
c: p1 p2 -> p3 p4
d: p4 -> p5
marking: p3=1 p4=1
```

3.2.1 Диаграмма маркировок

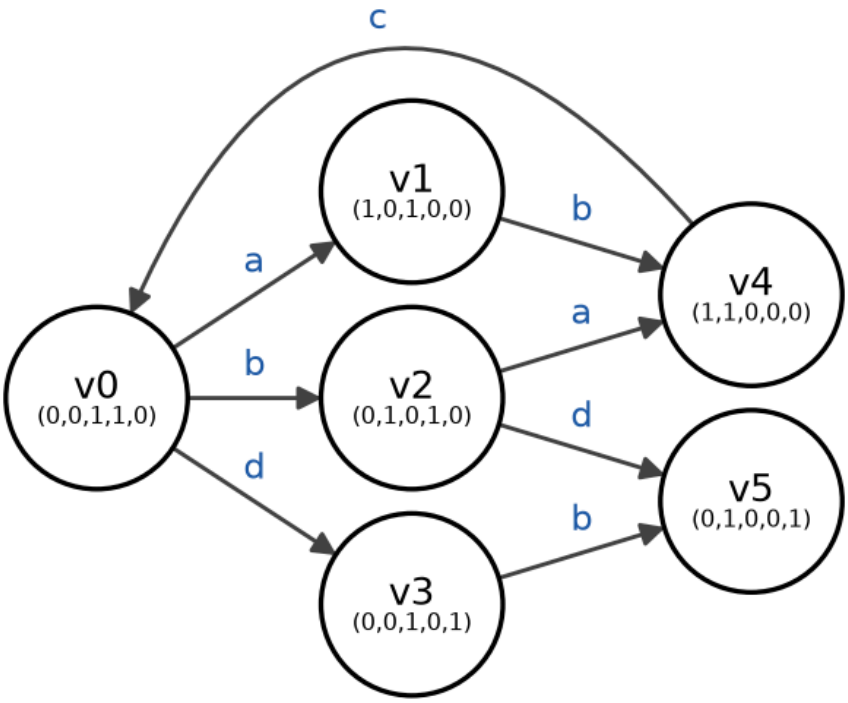


Рис. 3. Диаграмма маркировок левой сети: 6 вершин, 8 рёбер. Вершина v_5 — тупик.

Вершина	**Маркировка $(p_1, \ldots, p_5)$**	**Исходящие рёбра**
v_0	$(0, 0, 1, 1, 0)$	$a \to v_1$, $b \to v_2$, $d \to v_3$
v_1	$(1, 0, 1, 0, 0)$	$b \to v_4$
v_2	$(0, 1, 0, 1, 0)$	$a \to v_4$, $d \to v_5$
v_3	$(0, 0, 1, 0, 1)$	$b \to v_5$
v_4	$(1, 1, 0, 0, 0)$	$c \to v_0$
v_5	$(0, 1, 0, 0, 1)$	нет, тупик

3.2.2 Ответы на пункты задания

- **Достижимость.** Достижимы шесть маркировок из таблицы. Например, $(1, 1, 0, 0, 0)$ достижима (v_4), а $(0, 0, 2, 0, 0)$ — нет.
- **Параллельность.** В v_0 разрешены a, b и d. Пары a и b (разные входы p_4 и p_3) и b и d (входы p_3 и p_4) срабатывают параллельно. Пара a и d нет: обоим нужна фишка из p_4, а она одна. В других вершинах разрешённые переходы конфликтуют (v_2: a и d) или разрешён один.
- **Класс.** Это сеть свободного выбора. Единственная позиция с несколькими выходными переходами — p_4 (выходы a и d), и для каждого из них p_4 — единственный вход. Графом событий сеть не является: у p_4 два выходных перехода. Конечным автоматом тоже нет: у c два входа и два выхода.
- **Характеристики.** Границы во всех позициях равны 1, значит сеть безопасна. Есть один тупик v_5, поэтому сеть **не жива**. Из тупика нельзя вернуться в M_0, поэтому сеть **необратима**: например, из v_0 по d в v_3, затем по b в v_5, откуда выхода нет.

3.2.3 Сравнение с теорией

Каждый переход левой сети забирает и кладёт одинаковое число фишек (a, b и d переносят одну фишку, c берёт две и кладёт две), поэтому суммарное число фишек в сети постоянно и равно 2. Отсюда сразу следует ограниченность: больше 2 фишек не может быть ни в одной позиции. Диаграмма показывает точнее: в каждой позиции не больше 1, потому что обе фишки никогда не оказываются в одной позиции.

4 Как программа решает задачу

4.1 Сеть и маркировка: `net.rs`

```rust
pub type Marking = Vec<u32>;

pub struct Transition {
    pub name: String,
    pub source: Vec<usize>,   // входные позиции
    pub target: Vec<usize>,   // выходные позиции
}

pub struct Net {
    pub places: Vec<String>,
    pub transitions: Vec<Transition>,
}
```

Это единственное представление сети: общее правило срабатывания не зависит от конкретной сети, различаются только данные в списках `source` и `target`. Маркировка — вектор числа фишек, индекс совпадает с индексом в `Net::places`.

Функция	Что делает
`is_enabled(marking, t)`	Проверяет, что в каждой входной позиции `t` больше нуля фишек.
`enabled_transitions(marking)`	Список номеров разрешённых переходов в порядке их описания.
`fire(marking, t)`	Если `t` разрешён, копирует маркировку, вычитает по одной фишке из `source`, прибавляет по одной к `target` и возвращает новую маркировку. Иначе `None`. Исходная маркировка не меняется: диаграмме нужны копии состояний.
`can_fire_concurrently(marking, t, u)`	Считает для каждой позиции, сколько раз она входит в `source` переходов `t` и `u` вместе, и сравнивает с числом фишек. Параллельно, если фишек хватает везде.

`fire` сначала проверяет разрешённость и только потом вычитает: без проверки счётчик `u32` ушёл бы ниже нуля. Поэтому у функции нет предусловия, а вызывающему коду достаточно обработать `None`.

4.2 Диаграмма маркировок: `graph.rs`

```rust
pub struct ReachabilityGraph {
    pub markings: Vec<Marking>,   // вершина i — markings[i]; 0 — начальная
    pub edges: Vec<Edge>,         // Edge { from, transition, to }
    pub complete: bool,           // false: остановились на лимите
    index: HashMap<Marking, usize>,
}
```

`build(net, initial, limit)` — обход в ширину:

1. в очередь кладётся вершина 0 (начальная маркировка);
2. для вершины из очереди перебираются все разрешённые переходы, для каждого вычисляется новая маркировка;
3. если маркировка уже есть в `HashMap`, добавляется только ребро. Если её нет и вершин меньше `limit`, она получает следующий номер, попадает в очередь и в `HashMap`. Если вершин уже `limit`, флаг `complete` становится `false`, а ребро и маркировка пропускаются;
4. ребро добавляется на каждое срабатывание, в том числе ведущее в уже известную вершину (иначе диаграмма потеряла бы, например, ромб правой сети).

`HashMap` нужен, чтобы не заводить одну маркировку дважды и быстро отвечать «достижима ли маркировка» (`index_of`).

`concurrent_pairs(net)` перебирает все вершины и все пары разных переходов `t < u`, и возвращает те `(вершина, t, u)`, где `can_fire_concurrently` истинно. Если список не пуст, параллельное срабатывание в сети возможно.

4.3 Текстовый ввод: `parse.rs`

Формат такой:

```
t1: p1 -> p2 p3 p6   # переход: имя, входы, стрелка, выходы
marking: p1=1 p6=1   # начальная маркировка, остальные позиции — 0
```

- Позиции нумеруются по первому появлению в тексте, отдельно объявлять их не нужно.
- Пустые строки и всё после `#` игнорируются.
- Переход без входов или без выходов допускается. Переход без входов разрешён всегда, поэтому диаграмма такой сети остановится по лимиту.
- Позиция, которая встречается и во входах, и в выходах одного перехода, допустима.
- Кратные дуги запрещены: повтор позиции внутри списка входов или внутри списка выходов — ошибка. Так обе проверки (`is_enabled` по наличию фишки и `can_fire_concurrently` по подсчёту) остаются согласованными.
- Парсер собирает **все** ошибки, а не только первую, каждая с номером строки: нет `->`, повтор позиции, повтор имени перехода, неизвестная позиция или плохое число в маркировке, второй `marking`, пустой текст.

`parse_marking(text, net)` разбирает маркировку отдельно, для поля «достижима ли маркировка» в окне.

4.4 Класс и свойства: `analysis.rs`

`classify(net)` смотрит только на структуру и возвращает три флага:

- `state_machine`: у каждого перехода `source.len() == 1` и `target.len() == 1`;
- `marked_graph`: у каждой позиции ровно один входной и ровно один выходной переход;
- `free_choice`: для каждой позиции с несколькими выходными переходами все эти переходы имеют ровно один вход.

Флаги не исключают друг друга (граф событий всегда free-choice), и самый узкий класс окно составляет из них.

`analyze(graph, net)` возвращает `Properties`:

- `bounds` — максимум фишек в каждой позиции по всем вершинам;
- `deadlocks` — вершины без исходящих рёбер;
- `reversible` — обратный обход от вершины 0 по развёрнутым рёбрам достиг всех вершин;
- `live` — для каждого перехода обратный обход стартует из вершин, где есть ребро с ним, и достигает всех вершин;
- `complete` — копия флага диаграммы. Если он `false`, значения описывают только исследованную часть графа.

Обратимость — частный случай того же обхода: вместо множества вершин с ребром перехода стартом служит единственная вершина 0.

```rust
fn reached_backwards(preds: &[Vec<usize>], starts: &[usize]) -> Vec<bool> {
    // стек, предшественники по развёрнутым рёбрам
}
```

4.5 Окно: `canvas.rs` и `app.rs`

`Layout::circle` ставит позиции на внешнюю окружность, а переходы на внутреннюю (единственный переход — в центр). Координаты хранятся в долях окна от −1 до 1, поэтому при изменении размера окна рисунок не ломается. Перетаскивание меняет эти координаты.

`canvas::show_graph` рисует диаграмму маркировок. Слой вершины — длина кратчайшего пути от v_0, вершины одного слоя стоят столбиком, слои идут слева направо. Внутри вершины её номер и маркировка (только непустые позиции), над стрелкой имя перехода. Рёбра, которые возвращаются в тот же или более ранний слой (например t_4 в правой сети), рисуются дугами под рисунком, иначе они прошли бы сквозь вершины. Несколько рёбер между одной парой вершин раздвигаются, петля рисуется маленьким кружком над вершиной. Вершина, соответствующая текущей маркировке симуляции, подсвечена. Диаграмма из более чем 15 вершин не рисуется, вместо неё выводится сообщение с числом вершин.

`canvas::show` рисует сеть и возвращает кликнутый разрешённый переход.

- Позиции — кружки с числом фишек и подписью.
- Переходы — чёрточки: зелёные, если разрешены, серые иначе.
- Дуги — стрелки от границы одного узла до границы другого. Если между позицией и переходом дуги в обе стороны, они смещены на несколько пикселей и не накладываются.

`app.rs` держит текст, последнюю корректную сеть (`Loaded`), текущую маркировку и историю шагов для «Back». При каждом изменении текста сеть перечитывается, диаграмма и свойства пересчитываются, маркировка и раскладка сбрасываются. Если текст не разобрался, ошибки показываются красным, а на холсте остаётся последняя корректная сеть.

5 Тесты

`cargo test -p lab5` запускает 31 тест.

Группа	Что проверяет
`nets` (5)	Цикл правой сети через t_1, t_2, t_3, t_4 возвращает начальную маркировку, t_4 неразрешён, пока не закончены обе работы. Конфликт в p_4 левой сети. Цикл левой сети через b, a, c. `fire` на неразрешённом переходе даёт `None` и не меняет вход.
`graph` (7)	Правая сеть: 5 вершин, 6 рёбер. Левая сеть: 6 вершин, 8 рёбер, одна тупиковая вершина. Параллельные пары: у правой только t_2 и t_3 в v_1, у левой a с b и b с d в v_0. Две фишки в p_4 снимают конфликт a и d. Неограниченная сеть останавливается на лимите с `complete = false`. Лимит, равный числу вершин, не помечает граф неполным.
`parse` (8)	Тексты обеих сетей разбираются в те же сети, что и построенные вручную (сравнение по именам, потому что номера позиций зависят от текста). Комментарии, пустые строки, отсутствие маркировки. Переход без входов. Повтор позиции отвергается, позиция во входе и выходе допускается. Все ошибки собираются с номерами строк. `parse_marking`.
`analysis` (6)	Левая сеть: free-choice, безопасна, один тупик, необратима, не жива. Правая: marked graph, границы $(1, 1, 1, 2, 1, 1)$ в порядке появления позиций в тексте, обратима, жива. Конечный автомат. Конфликт с дополнительным входом — не free-choice. Сеть без тупиков, но не обратимая и не живая. Неограниченная сеть помечена неполной.
`canvas` (5)	Круговая раскладка: узлы различны и лежат внутри единичного круга. Один переход в центре. Выход линии из чёрточки через ближайшую сторону. Слои диаграммы маркировок: $(0, 1, 2, 2, 3)$ у правой сети и $(0, 1, 1, 1, 2, 2)$ у левой. Послойные позиции: столбцы по слоям, общий шаг по вертикали, всё внутри единичного квадрата.

Результаты обеих диаграмм (вершины и рёбра) проверены независимой реализацией на Python: те же вершины, те же рёбра, те же номера в порядке обхода в ширину. Окно запускал пользователь (отзыв: «выглядит норм»); отрисовка автоматическими тестами не покрыта.

6 Допущения и ограничения

- **Схема прочитана по картинке.** Рисунок в PDF прочитан как изображение. Если стрелки идут иначе, меняются тексты в `nets.rs`.
- **Кратность дуг 1.** Обе сети из задания такие. Дуги кратности 2 и больше запрещены при вводе, а не поддержаны.
- **Лимит 10 000 вершин.** Он защищает от бесконечной диаграммы. Остановка по лимиту не доказывает неограниченность сети, окно показывает предупреждение, а свойства считаются только для исследованной части.
- **Параллельность через пары.** Проверяются только пары разных переходов (этого достаточно для ответа «возможно ли вообще»). Срабатывание одного и того же перехода дважды одновременно (нужно две фишки на входе) не рассматривается.
- **«Теоретические характеристики» не определены в задании.** Я трактовал их как свойства, известные по теории для данных классов (раздел 3). Что именно имел в виду преподаватель, лучше уточнить.
- **Классы.** Программа определяет три структурных признака и общий класс. Если в методичке есть другие классы, их нужно добавить в `classify`.
- **Раскладка сети.** Круговая, не оптимизирована: на больших сетях дуги пересекаются, и узлы нужно разводить вручную перетаскиванием.
- **Диаграмма маркировок в окне.** Рисуется только до 15 вершин, узлы не перетаскиваются. Ребро в вершину того же слоя рисуется как возвратное (дугой под рисунком), для редких случаев этого достаточно.